

WF-TFC: An Open-World Few-Shot Anonymous Website Fingerprinting via Time-Frequency Consistency

Xiaolan Zhu¹, Junfeng Wang², *Member, IEEE*, Wenhan Ge³, Yizhao Huang⁴, and Tingting Lu⁵

Abstract—While Tor provides strong anonymity, it also facilitates the concealment of malicious activities, which poses a significant challenge to cybersecurity surveillance. As an effective anti-anonymity technique, Website Fingerprinting(WF) enables the inference of which websites a user is visiting, thereby uncovering potential attacker activities. State-of-the-art(SOTA) methods have demonstrated remarkable effectiveness. However, a large number of labeled traffic is required to ensure effectiveness, and without timely updates, these models will encounter serious challenges of concept drift due to the dynamic nature of website content and network conditions. The core reasons lie in the independently and identically distributed assumption, while in challenging open-world scenarios, the long-term spatial and temporal dynamics complicates data consistency and effective knowledge transfer. To address these issues, this paper presents WF-TFC, an open-world few-shot anonymous WF model via self-supervised contrastive learning and time-frequency consistency. It aligns time- and frequency-based representations in the latent time-frequency space, enhancing the sustained effectiveness of inherent patterns across various websites. Consequently, it accommodates diverse few-shot target domains with varying dynamics, facilitating data consistency and knowledge transfer in unobserved long-term temporal and spatial environments. For instance, with only 5 traces per website, WF-TFC achieves 92.62% accuracy on traces collected six weeks after pre-training, exceeding the SOTA(i.e., NetCLR) by 2.12%. On similar but mutually exclusive traces, it attains an F1 score of 87.20%, surpassing the SOTA by 6.12%.

Index Terms—Website fingerprinting, self-supervised contrastive learning, time-frequency consistency, few-shot learning.

I. INTRODUCTION

IN RECENT years, anonymous network communication has been increasingly abused by malicious attackers. Tor network [1] is one of the most widely used low-latency anonymous communication systems, with over three million

daily users [2]. This anonymity serves as a breeding ground for illicit purposes, facilitating activities such as drug trafficking, arms trade, Bitcoin transactions, and various cyberattacks [3]. According to Bitcoin technology company Bitfury, there has been a significant increase in the value of Bitcoin transactions on anonymous networks, with annual growth reaching 65% and an overall rise of over 340% in the past three years [4]. Additionally, hundreds of malwares exploit Tor to hide their presence and hinder Command and Control(C&C) takedown operations [5], ransomware like WannaCry and Skynet exemplifies this to deploy C&C servers and evade law enforcement detection, posing a great threat to cybersecurity.

Previous studies have shown that Tor is susceptible to WF [6], [7], [8], [9], [10], an anti-anonymity attack that can identify accessed websites. Traffic patterns between clients and servers vary across websites, adversaries can exploit these distinct sequences, referred to as “fingerprints”, to infer users’ anonymous browsing activity. These fingerprints include features like packet timestamps, directions, and sizes, collected by passive adversaries such as Internet Service Providers(ISPs) or surveillance agencies [11], [12], [13]. Since adversaries’ inherent nature does not easily change with users’ intentions or actions, analyzing such fingerprints poses fewer privacy challenges than directly handling raw communication content or payloads [14], [15].

Traditional WF methods mainly focus on manually engineered features and apply machine learning algorithms for classification [6], [7], [16], [17], [18], [19], [20]. Advanced WF models that utilize deep neural networks have demonstrated improved performance [8], [9], [21], [22], such as DF [9] reaching 98% accuracy in closed-world settings with only packet direction sequences. In open-world scenarios, the dynamic changes in website content and network conditions cause significant variations in traffic distribution for the same website over time, resulting in concept drift [23]. As the fingerprints that distinguish different websites continuously change, previously used features may become less ineffective, leading to a decline in the model’s performance. Therefore, a large amount of labeled traces and frequent retraining is required by these methods to against concept drift. Moreover, they are often criticized for being impractical in realistic and dynamic open-world scenarios, as they assume training and testing data are independently and identically distributed. In reality, it is challenge for an adversary to fully replicate a user’s network conditions, browser settings, or system

Received 22 October 2024; revised 1 April 2025; accepted 12 June 2025. Date of publication 18 June 2025; date of current version 26 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U24B20147 and Grant U2133208; and in part by the Major Science and Technology Special Project of Sichuan Province under Grant 2024ZHCG0195, Grant 2024ZDZX0044, and Grant 2024ZYD0269. The associate editor coordinating the review of this article and approving it for publication was Prof. Kun Sun. (Corresponding author: Junfeng Wang.)

Xiaolan Zhu and Yizhao Huang are with the National Key Laboratory of Fundamental Science on Synthetic Vision, Sichuan University, Chengdu 610065, China.

Junfeng Wang, Wenhan Ge, and Tingting Lu are with the College of Computer Science, Sichuan University, Chengdu 610065, China (e-mail: wangjf@scu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3581092

configurations, making it more difficult to regularly collect extensive traffic. On the other hand, discrete changes in traces can impede threshold-based fully supervised learning from achieving adequate short-term training, highlighting the urgent need for methods with relative measurement capabilities, with few-shot learning [24], [25] emerging as a promising solution.

This work aims to alleviate the mentioned issues by introducing WF-TFC, a novel open-world few-shot anonymous WF approach that utilizes self-supervised contrastive learning and time-frequency consistency to enhance generalized representations across diverse website traffic. It involves task-agnostic pre-training to extract robust features from historical traces, followed by task-specific fine-tuning to tailor them for downstream tasks with limited traces. Extensive experiments are conducted in both closed- and open-world anonymous scenarios, demonstrating the effectiveness and superiority of WF-TFC in challenging few-shot settings. Consequently, WF-TFC effectively accommodates diverse few-shot target domains with varying dynamics, facilitating the model's knowledge transferability and data compatibility in unobserved long-term temporal and spatial network environments. It innovatively introduces frequency domain analysis, along with a novel time-frequency consistency, achieving superior classification performance over existing methods. To sum up, the main contributions of our work are as follows:

- 1) We present WF-TFC, a novel open-world few-shot anonymous WF approach based on self-supervised learning and time-frequency consistency pre-training. It effectively aligns time- and frequency-domain embeddings within the latent time-frequency space, enhancing the generalizability of website traffic representations, while reducing the reliance on large amounts of labeled traffic.
- 2) We explore effective augmentation methods for traces across different websites to simulate variations in dynamic website content and diverse network environments. By considering both time- and frequency-domain perspectives, traces are augmented to capture the inherent characteristics in unobserved long-term temporal and spatial dynamics, enhancing the model's transferability to diverse few-shot tasks.
- 3) We perform extensive experiments in both closed- and open-world anonymized scenarios, including the more challenging 1-shot and 2-shot settings, demonstrating the effectiveness and superiority of WF-TFC under realistic few-shot settings.

The rest of this paper is organized as follows: Section II provides a brief introduction to the background, while Section III reviews the related work. In Section IV, we present a detailed description of WF-TFC. Section V shows the experimental setup and results, followed by a discussion in Section VI. Finally, we conclude the WF-TFC in Section VII.

II. BACKGROUND

In this section, we first give the threat model of WF. Then, the concept of contrastive learning is briefly introduced, followed by an overview of signal processing theory.

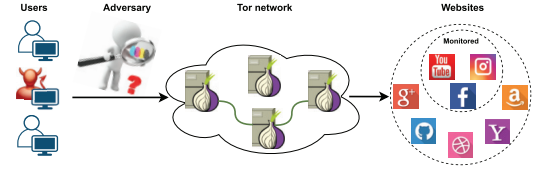


Fig. 1. The threat model of WF.

A. Threat Model

As illustrated in Figure 1, this model outlines the scenario in which the adversary analyzes traffic between a user, whether engaged in normal or malicious activities, and the server to infer anonymous websites through fingerprints. We consider a passive, local adversary who monitors traffic between the user and the Tor entry relay without actively interfering. Examples of these adversaries include a local ISP, a compromised network node, or a LAN administrator.

Closed-World vs Open-World: WF is typically evaluated in two scenarios: closed- and open-world. In the closed-world scenario, the adversary assumes that the user can only visit a predefined set of monitored websites. With this assumption, she can collect traffic fingerprints for all target websites, enabling precise classification. In contrast, the open-world scenario reflects a more realistic setting, where users can access both monitored and unmonitored websites. The key challenge for the adversary is not only identifying monitored websites but also distinguishing them from unmonitored ones, increasing identification difficulty.

Conversely, anti-anonymity defenses like WTF-PAD [26], FRONT [27] and TAMARAW [28], aim to obscure traffic patterns and hinder WF identification by injecting dummy packets, introducing noise, or altering traffic characteristics. Lightweight defenses like WTF-PAD and FRONT balance privacy and efficiency through adaptive techniques. WTF-PAD injects dummy packets only during traffic gaps to minimize bandwidth overhead, while FRONT employs randomized packets that adjust to observed traffic patterns. TAMARAW, a typical heavyweight method, uses fixed-rate, fixed-length dummy packets, offering high effectiveness but introducing significant bandwidth overhead.

B. Contrastive Learning

Contrastive learning is a form of self-supervised learning designed to learn effective feature representations by comparing the similarities between samples [29]. Its objective is to bring similar samples closer together in the embedding space while pushing dissimilar ones farther apart. To achieve this, various data augmentation techniques are applied to each data sample, generating two distinct views for comparison. Thus, the model learns to extract meaningful and discriminative features through a contrastive loss function [30], which can be formulated as:

$$L = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^N \exp(\text{sim}(z_i, z_k)/\tau)} \quad (1)$$

where z_i and z_j denote the embeddings of two augmented views of the same instance, with $\text{sim}(z_i, z_j)$ quantifying their

similarity(e.g., cosine similarity). The temperature parameter τ adjusts the sharpness of the distribution, and N is the total number of samples in the batch, with the denominator aggregating all negative samples.

C. Signal Processing Theory

Signal processing theory is a branch of applied mathematics and engineering that focuses on analyzing, manipulating, and transforming signals [31], [32]. A signal is any measurable quantity that conveys information, such as electrical, acoustic, optical, or biological. It encompasses both continuous and discrete representations, enabling the manipulation and analysis of signals across various domains, including time and frequency. Time-domain analysis offers insights into a signal's behavior over time, while frequency-domain analysis reveals its underlying frequency components.

In the time domain, signals are represented as functions of time, reflecting their variation over time. For network traffic, data flow can be expressed in the time domain, with metrics like packet arrival times, transmission durations, and network delays fluctuating over time. In contrast, frequency-domain analysis decomposes the signal into its constituent frequencies, revealing dominant frequencies, harmonics, and periodic components. Similarly, the frequency domain provides insights into traffic dynamics, uncovering patterns such as periodicity, burstiness, and spectral properties. A signal in the time domain can be transformed into the frequency domain using techniques like the Fast Fourier Transform(FFT) [33]. Together, both domains offer complementary perspectives, providing a comprehensive view of network behavior.

III. RELATED WORK

This section reviews related work, which is broadly categorized into methods based on manual feature design and those leveraging automated feature extraction. A summary of their limitations is provided at the end.

A. Manual Feature Design-Based Methods

Since Herrmann et al. [16] first apply WF to the Tor network in 2009, various machine learning algorithms have been extensively utilized in this field. These methods involve manually extracting meaningful features to train the classification model for classification. Panchenko et al. [6] use Support Vector Machine(SVM), achieving notable classification accuracy. Cai et al. [17] leverage Damerau-Levenshtein distance along with an SVM classifier to evaluate the similarity of website traces. Wang and Goldberg [7] further improve this by using Tor cells instead of TCP/IP packets. CUMUL is later introduced in [19], using an SVM classifier with 104 features derived from directional packet length sequences to enhance classification. Wang et al. [18] improve K-Nearest Neighbors(KNN) by dynamically adjusting feature weights and extracting nearly 4,000 traffic features, obtaining high accuracy in closed-world scenarios. Hayes and Danezis [20] introduce K-FP, which uses Random Forests to extract the top 150 features based on importance scores. In closed-world settings, the output of Random Forest is used for classification, while in open-world

scenarios, KNN classifies test samples by comparing them to the K nearest training traces.

B. Automated Feature Extraction-Based Methods

Advanced deep learning methods automatically extract features, greatly improving WF identification performance. Abel and Goto [21] are the first to apply deep learning to WF, using Stacked Denoising Autoencoders(SDAE) to reduce overfitting. Rimmer et al. [8] employ three deep learning models: SDAE, Convolutional Neural Network(CNN), and Long Short-Term Memory(LSTM). They contribute the largest WF dataset and show that SDAE outperforms the best traditional manual feature-based approach, K-FP. The DF method in [9] differs from Rimmer et al.'s CNN by applying pooling layers only after two convolutional layers, which leads to a deeper network with improved feature extraction. It further mitigates overfitting with Batch Normalization and Dropout, increases filter numbers in deeper layers to capture complex features, and uses the ELU activation function near the input to minimize information loss. Zhou et al. [22] propose WF-Transformer, which uses Transformer to extract temporal features from traffic, achieving superior performance with shorter trace lengths.

Although effective, a large amount of labeled traffic and frequent retraining are required to maintain accuracy and guard against concept drift. Juárez et al. [34] find that the accuracy of a KNN classifier dropped from 80% to 30% within 10 days, while Rimmer et al. [8] observe a decline from 95% to 81% in their SDAE classifier within 28 days. Thus, classifiers may remain effective for only a few weeks. Regularly collecting large amounts of traffic traces is essential but can be time-consuming and labor-intensive. For example, capturing 500,000 traces takes a single terminal resource adversary 250 days [35]. Moreover, they assume that training and testing data are independently and identically distributed, limiting their knowledge transferability in open-world long-term spatio-temporal network environments. Juárez et al. [34] also explore how variations in Tor Browser Bundle(TBB) versions, configurations, and network locations affect classification accuracy. They find that different TBB versions can reduce accuracy by over 50%, while different configurations and network locations can introduce variations of up to 11.7% and nearly 60%, respectively. This issue is especially pronounced in few-shot scenarios, where limited data exacerbates data compatibility challenges, diminishing the model's generalization and classification performance.

There are recent studies have explored limited-data WF attacks to mitigate concept drift. Sirinam et al. [35] propose Triplet Fingerprinting(TF) based on N-shot learning. They first pre-train a feature extractor using a Triplet Network [36], then fine-tune it with KNN to classify target websites. Chen et al. [37] use harmonious data augmentation to expand limited dataset and enhance the accuracy of deep learning models, but it performs poorly with fewer traces. Bhat et al. [38] design Var-CNN, based on ResNet-18, using dilated causal convolutions [39] and incorporating seven cumulative statistical features to improve accuracy. Chen et al. [40] further propose TLFA, which involves pre-training an embedding model on a large set of labeled traces and fine-tuning it on a

limited amount of target traffic. Lu et al. [41] extract features from multi-temporal traffic during pre-training to generate robust representations. Tan et al. [42] propose a collusion-based few-shot WF attack that enhances user-side adaptability by integrating multiple attackers. Cherubin et al. [43] use features from Tor exit relay traces for a more realistic WF, but faced poor classification accuracy, with data not shared due to privacy concerns. Bahramali et al. [44] introduce NetAugment to augment network traces, and their NetAugment-based model (NetCLR) achieves up to 80% accuracy using just five traces from each website. Ding and Hu [45] propose LRCT, which combines local Recurrent Neural Network (RNN) and CNN to extract fine-grained spatio-temporal features from cell sequences, enabling the capture of rich global features.

On the whole, the manual feature design-based methods rely on meticulously designed features, which may lead to failures in certain cases due to the unreliability of flow statistical information. The automated feature extraction-based models significantly enhance performance by leveraging the powerful representational capabilities of deep neural networks. However, a large amount of labeled traffic and frequent retraining are required to ensure the sustained effectiveness of the model. Moreover, they are often criticized for being less practical in dynamic open-world scenarios, as they assume that training and testing data are independently and identically distributed. Recent studies in few-shot WF have largely mitigated these issues, but they only consider the time-domain analysis of traffic, and the lack of a more effective pre-training strategy still leads to suboptimal classification performance.

IV. METHODOLOGY

In this section, we first introduce our motivation and the problem formulation of few-shot WF identification. Then, we present the design details of WF-TFC, beginning with a framework overview before delving into its individual components.

A. Motivation

In open-world scenarios, dynamic changes in website content and network conditions lead to concept drift, requiring deep learning models to continuously collect large labeled traffic data and retrain to maintain performance. Inspired by advanced self-supervised learning techniques in computer vision and natural language processing [46], [47] [48], data augmentation techniques offer a promising solution. However, they cannot be directly applied to network traffic due to its dynamic nature and the strong correlations within packet sequences.

Based on user-server interaction characteristics, outgoing traffic consists of smaller data packets generated by user requests, while incoming traffic includes larger responses from the server, such as images, text, and videos. Figure 2 illustrates the mean and standard deviation of the number of cells from 50 randomly selected websites, showing that incoming cells significantly outnumber outgoing cells and both vary dynamically. To replicate this variation under unobserved conditions, we randomly modify incoming and outgoing flows based on bursts representing clustering in the same direction. Thus,

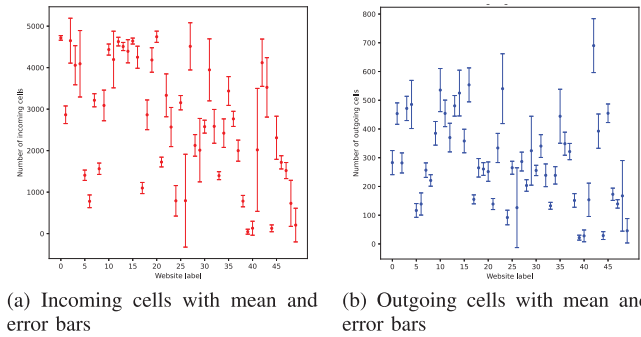


Fig. 2. Mean and standard deviation of the number of cells for 50 randomly selected websites.

self-supervised learning can be further used to effectively learn robust representations, while minimizing the reliance on extensive labeled traffic.

Additionally, time-domain analysis captures instantaneous changes and local features of network traffic, while frequency-domain analysis reveals traffic behaviors that are not directly observable in the time domain [49]. Grounded in signal processing theory, this analysis ensures consistency across data distributions. Together, they provide a comprehensive understanding of traffic behavior and serve as an inductive bias to pre-train models for few-shot WF identification.

B. Problem Formulation

Before introducing WF-TFC, we first present the problem addressed in this paper and explain how it effectively addresses these issues.

In this work, we focus on the challenge of anonymous WF identification, particularly in open-world few-shot settings, where only a limited number of labeled traces are available for each website. This issue is further exacerbated by dynamic website content and fluctuating network environments, resulting in concept drift and shifts in data distribution over time, ultimately causing long-term temporal and spatial variations in traffic. Concept drift is a phenomenon where the statistical properties of data change over time, causing previously learned features to become ineffective. Data distribution shift refers to changes in data distribution across domains or time periods, especially in open-world scenarios where new, unseen websites emerge. Both can lead to performance degradation, requiring models to be retrained and updated in a timely manner. In few-shot settings, the challenges posed by concept drift and data distribution variation are more pronounced due to limited labeled traffic, making it harder for model to adapt to new patterns over time.

WF-TFC is specifically designed to address the challenges mentioned above in open-world few-shot settings. By incorporating both time- and frequency-domain perspectives, traces are first augmented to capture the inherent characteristics of unobserved long-term temporal and spatial network environments. Then, self-contrastive learning and time-frequency consistency are employed to pre-train the feature embedding model, capturing the semantic invariance and common features

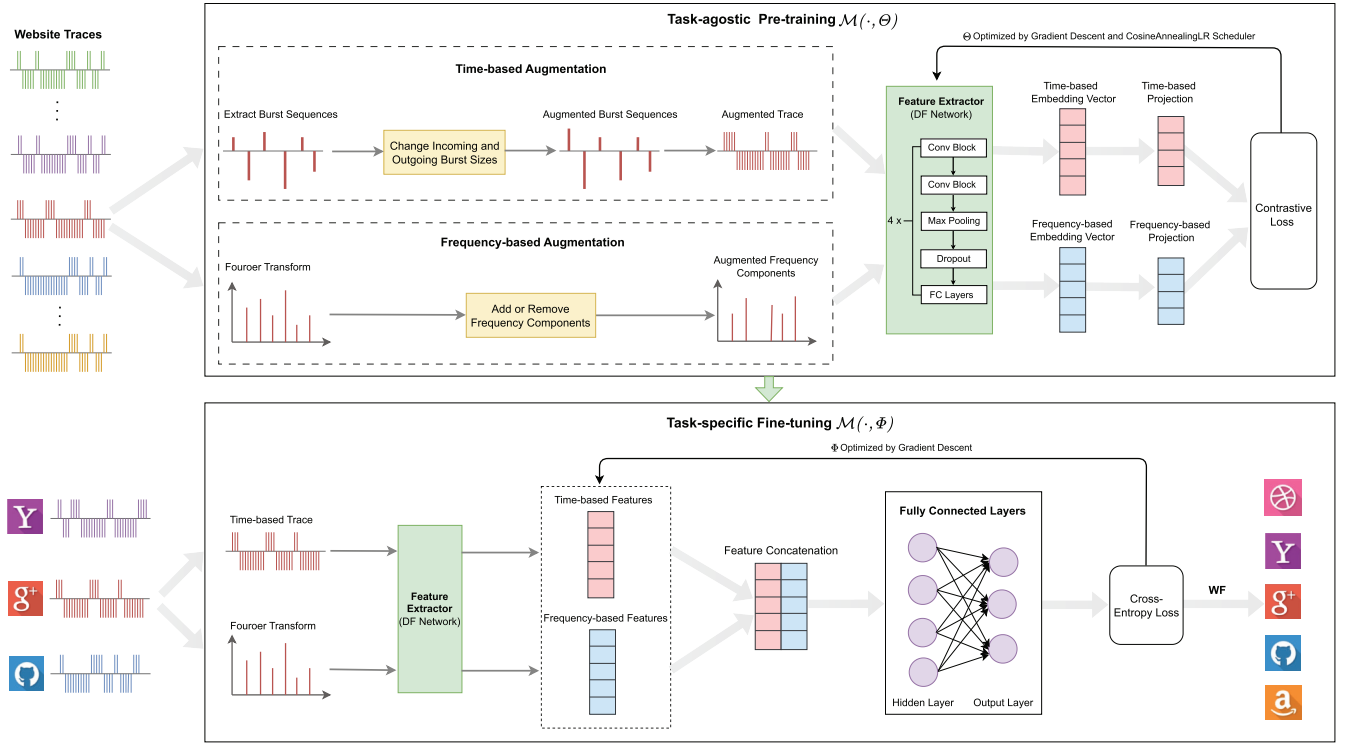


Fig. 3. Schematic overview of WF-TFC.

within traces. Finally, the model is fine-tuned with a few labeled traces per website from diverse few-shot tasks.

Given a pre-training dataset $D^{pre} = \{x_i^{pre} \mid i = 1, 2, \dots, N_p\}$ consisting of N_p unlabeled traces, where each trace x_i^{pre} contains 5000 cells represented by their direction. Let $D^{tune} = \{(x_i^{tune}, y_i) \mid i = 1, 2, \dots, N_t\}$ be the target dataset of labeled traces, where each trace x_i^{tune} corresponds to a website $y_i \in \{1, 2, \dots, C\}$, with C representing the number of websites in D^{tune} . Notably, D^{tune} is independent of D^{pre} . In this work, we use time-frequency consistency and trace augmentation during pre-training. Here, x_i also denoted as x_i^T , represents an input trace, and x_i^F represents the discrete frequency spectrum of x_i^T . The augmented x_i^T and x_i^F are referred to as $\tilde{x}_i^T$ and $\tilde{x}_i^F$, respectively.

In few-shot scenarios, the number of traces N_t in D^{tune} is much smaller than N_p in D^{pre} ($N_t \ll N_p$). The goal is to pre-train a generalized model $\mathcal{M}$ on D^{pre} with parameters Θ and then fine-tune it on D^{tune} , transitioning from $\mathcal{M}(\cdot, \Theta)$ to $\mathcal{M}(\cdot, \Phi)$ to generate robust and generalizable representations $z_i^{tune} = \mathcal{M}(x_i^{tune})$. Specifically, z_i^T and z_i^F are the time- and frequency-based representations of x_i , whether from D^{pre} or D^{tune} , while $\tilde{z}_i^T$ and $\tilde{z}_i^F$ represent their augmented versions.

C. Framework Overview

As shown in Figure 3, the proposed WF-TFC consists of two phases: task-agnostic pre-training and task-specific fine-tuning. Task-agnostic pre-training equips the model with robust, generalized representations, while task-specific fine-tuning tailors it to specific few-shot target tasks. In the first phase, the task-agnostic feature embedding model $\mathcal{M}(\cdot, \Theta)$ is

pre-trained using self-supervised learning without requiring any labeled data. During fine-tuning, the model $\mathcal{M}(\cdot, \Phi)$ is trained to adapt to diverse downstream tasks with a few labeled traces per website, enhancing overall performance.

Building on prior works [35], [40], [44], WF-TFC also utilizes the DF deep network [9] as the backbone. We further enhance the pre-trained representations by adding a projection layer, with Θ optimized using a contrastive loss based on time-frequency consistency. During fine-tuning, the pre-trained model transfers knowledge from D^{pre} to initialize the target model on D^{tune} , adapting it to various few-shot tasks. The invariant nature of time-frequency consistency bridges the knowledge gap between D^{pre} and D^{tune} , even with major differences such as varying distributions, data collected over multiple years, or a large number of unmonitored websites.

D. Task-Agnostic Pre-Training

Models trained on limited traffic often lack reliability, making pre-training on large datasets crucial for generalization. Self-supervised contrastive learning leverages unlabeled data to learn robust, discriminative features, providing strong support for few-shot scenarios. Our task-agnostic pre-training model $\mathcal{M}$ comprises four components: a time encoder G_T , a frequency encoder G_F , and two cross-space projectors R_T and R_F , as is shown in Figure 4. G_T captures time-domain patterns, while G_F focuses on frequency-domain representations. The projectors R_T and R_F align the outputs of their respective encoders across both domains. This design enables $\mathcal{M}$ to effectively integrate time and frequency information, along with local and global features, enhancing its ability to capture inherent traffic patterns.

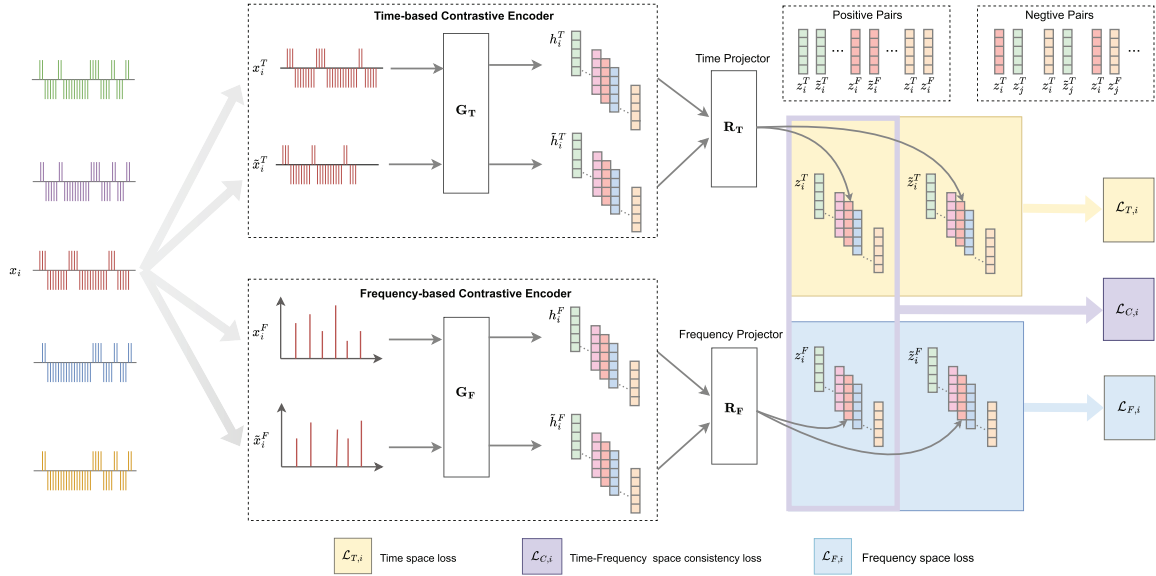


Fig. 4. Architecture of our task-agnostic pre-training model.

1) *Trace Augmentation*: To simulate variations in dynamic website content and different network environments, we consider both time- and frequency-based perspectives.

a) *Time-based augmentation*: From the time-domain analysis, a website trace is augmented by simultaneously modifying the sizes of incoming and outgoing burst cells. We randomly adjust their sizes by increasing or decreasing the values to reflect the characteristics of traffic during user-server interactions. The detailed implementation of this augmentation process is shown in Algorithm 1.

Initially, the burst sequence B is extracted from x_i , after which the sizes of the incoming and outgoing bursts are adjusted accordingly. The direction of modification determined probabilistically by a random variable p , which dictates whether the burst size increases or decreases. Each modification is constrained by the thresholds th_{in} and th_{out} . Specifically, the incoming burst sizes are adjusted using the hyperparameters $r_{inc,in}$ and $r_{dec,in}$, while the outgoing burst sizes are modified with $r_{inc,out}$ and $r_{dec,out}$. Consequently, the augmented burst sequences are reconstructed into the original cells, generating the augmented website trace $\tilde{x}_i^T$.

b) *Frequency-based augmentation*: Spectral analysis is employed to identify periodic variations and fluctuations from a frequency-domain perspective. Since each frequency component represents a bias function with specific frequency and amplitude, we perturb the spectrum by randomly adding or removing components. Algorithm 2 outlines the implementation of the frequency-based website trace augmentation.

The frequency representation x_i^F of the input signal x_i is first obtained using FFT to compute the magnitude spectrum, followed by a logarithmic transformation to stabilize numerical values. Next, frequency augmentation is applied based on the random variable p . If $p \geq 0.5$, a set of r_{remove} components are randomly selected and their amplitudes are set to 0. Otherwise, the amplitudes of the chosen frequency components are randomly increased by $r_{add} \times A_max$, where A_max represents

the maximum amplitude. Both r_{add} and r_{remove} are predefined hyperparameters that control the modification ratio. Finally, the augmented frequency representation $\tilde{x}_i^F$ is obtained.

2) *Time-Based Contrastive Encoder*: For a given input trace x_i^T , a time-based augmentation generates $\tilde{x}_i^T$. Both x_i^T and $\tilde{x}_i^T$ are fed into the contrastive time encoder G_T , producing high-dimensional representations $h_i^T = G_T(x_i^T)$ and $\tilde{h}_i^T = G_T(\tilde{x}_i^T)$. This encoder brings similar traces closer in the representation space while pushing dissimilar ones apart, enhancing differentiation and classification. Then, h_i^T and $\tilde{h}_i^T$ are passed through the time projector R_T , yielding $z_i^T = R_T(h_i^T)$ and $\tilde{z}_i^T = R_T(\tilde{h}_i^T)$, mapping high-dimensional representations to a lower-dimensional space for effective learning via contrastive loss.

Since $\tilde{x}_i^T$ is an augmented version of x_i^T , after passing through G_T and R_T , their representations are expected to be closer in the time embedding space. Conversely, the distance between x_i^T and any other sample x_j^T , or its augmented version $\tilde{x}_j^T$, will be larger. Thus, $(x_i^T, \tilde{x}_i^T)$ forms a positive pair, while (x_i^T, x_j^T) or $(x_i^T, \tilde{x}_j^T)$ are negative pairs, allowing the pre-trained model to enhance classification by maximizing positive pair similarity and minimizing negative pair similarity. To achieve this, we apply contrastive loss to learn meaningful representations. Specifically, the time-based contrastive loss for x_i^T is defined as follows:

$$\mathcal{L}_{T,i} = -\log \frac{\exp(\text{sim}(z_i^T, \tilde{z}_i^T) / \tau)}{\sum_{x_j \in D^{pre}} \mathbb{1}_{[i \neq j]} \exp(\text{sim}(z_i^T, R_T(G_T(x_j^T))) / \tau)} \quad (2)$$

where, $\text{sim}(u, v) = u^T v / \|u\| \|v\|$ represents the cosine similarity between the L2-normalized embeddings u and v , and τ is a temperature parameter that controls the concentration of the similarity distribution. The indicator function $\mathbb{1}_{[i \neq j]}$ is typically used to distinguish between positive and negative pairs, ensuring that only negative pairs contribute to the denominator

Algorithm 1 Time-Based Website Trace Augmentation**Input:**

x_i //Vector of cells direction of a flow
 $r_{inc,in}$ //Rate of increasing incoming burst sizes
 $r_{dec,in}$ //Ratio of decreasing incoming burst sizes
 $r_{inc,out}$ //Ratio of increasing outgoing burst sizes
 $r_{dec,out}$ //Ratio of decreasing outgoing burst sizes
 $p \sim U(0, 1)$ //Randomly decide to increase or decrease burst size
 th_{in} //Minimum threshold for non-zero cells in an incoming burst
 th_{out} //Minimum threshold for non-zero cells in an incoming burst

Output: $\tilde{x}_i^T$ **Step 1:** Extract burst sequences $B \leftarrow \text{ExtractBursts}(x_i)$ **Step 2:** Modify incoming burst sizesInitialize $B_{in} \leftarrow []$ **if** $p \geq 0.5$ **then**

Increase incoming burst sizes

foreach $b \in B$ **do****if** $b \leq -th_{in}$ **then** $b \leftarrow b \times (1 + U(0, 1) \cdot r_{inc,in})$ **end**Append b to B_{in} **end****end****else**

Decrease incoming burst sizes

foreach $b \in B$ **do****if** $b \leq -th_{in}$ **then** $b \leftarrow b \times (1 - U(0, 1) \cdot r_{dec,in})$ **end**Append b to B_{in} **end****end****Step 3:** Modify outgoing burst sizesInitialize $B_{out} \leftarrow []$ **if** $p \geq 0.5$ **then**

Increase outgoing burst sizes

foreach $b \in B_{in}$ **do****if** $b > th_{out}$ **then** $b \leftarrow b \times (1 + U(0, 1) \cdot r_{inc,out})$ **end**Append b to B_{out} **end****end****else**

Decrease outgoing burst sizes

foreach $b \in B_{in}$ **do****if** $b > th_{out}$ **then** $b \leftarrow b \times (1 - U(0, 1) \cdot r_{dec,out})$ **end**Append b to B_{out} **end****end****Step 4:** Reconstruct the augmented cell sequence $\tilde{x}_i^T \leftarrow \text{ConvertBurstsToCells}(B_{out})$ **return** $\tilde{x}_i^T$ **Algorithm 2** Frequency-Based Website Trace Augmentation**Input:**

x_i //Vector of cells direction of a flow
 r_{remove} //Ratio of frequency components to be removed
 r_{add} //Ratio of frequency components to be increased
 A_{max} //Maximum amplitude
 $p \sim U(0, 1)$ //Randomly decide to remove or add frequency components

Output: $\tilde{x}_i^F$ **Step 1:** Compute frequency representation
 $x_i^F \leftarrow FFT(x_i)$ $A \leftarrow |x_i^F|$ $A_{half} \leftarrow A[: N//2]$ $A_{log} \leftarrow \log(A_{half} + \epsilon)$
Step 2: Apply frequency augmentation**if** $p \geq 0.5$ **then** $mask \leftarrow \text{Uniform}(0, 1) < r_{remove}$ $\tilde{x}_i^F \leftarrow A_{log} \times mask$ **end****else** $mask \leftarrow \text{Uniform}(0, 1) < r_{add}$ $\tilde{x}_i^F \leftarrow A_{log} + mask \times (r_{add} \times A_{max})$ **end****return** $\tilde{x}_i^F$

in the contrastive loss. This ensures similar samples remain close in the time-based feature space after dimensionality reduction, while dissimilar ones are pushed apart, preserving distinctiveness.

3) *Frequency-Based Contrastive Encoder*: The frequency domain representation x_i^F from x_i^T is obtained using Fourier Transformation [33]. This enhances traffic pattern extraction by leveraging frequency domain information, reducing information loss compared to using time-based traces alone [50]. The frequency representation of x_i^T can be obtained as follows:

$$F_i = \mathcal{F}(f_i) \quad (3)$$

$$F_{ik} = \sum_{n=0}^{N-1} f_{in} \cdot e^{-i2\pi(k-1)n/N} \quad (4)$$

Here, f_i denotes x_i^T , and F_k represents the corresponding frequency components. The index k ranges from 1 to 5000, mapping the frequency components from 0 to $N-1$, where N is the total number of sampling points.

Next, to convert the complex frequency domain representations F_{ik} into real numbers, their magnitudes are calculated. For simplicity, we then represent these magnitudes in a coordinate plane representation:

$$F_{ik} = a_{ik} + jb_{ik} \quad (5)$$

$$a_{ik} = \sum_{n=0}^{N-1} f_{in} \cdot \cos\left(\frac{2\pi(k-1)n}{N}\right) \quad (6)$$

$$b_{ik} = \sum_{n=0}^{N-1} f_{in} \cdot \sin\left(\frac{2\pi(k-1)n}{N}\right) \quad (7)$$

where a_{ik} and b_{ik} represent the real and imaginary parts of the complex number, respectively. The modulus of F_{ik} is calculated as p_{ik} using formula 8, and the first half is selected

to form the vector p_i since the Discrete Fourier Transform [51] ensures symmetry between the two halves. Thus, we obtain:

$$p_{ik} = \sqrt{a_{ik}^2 + b_{ik}^2} \quad (8)$$

$$p_i = [p_{i1}, p_{i2}, \dots, p_{i\frac{N}{2}}] \quad (9)$$

To ensure numerical stability and prevent floating-point overflow during model training [52], a logarithmic transformation is applied to p_i , with a constant C used to adjust the range of the frequency components. The adjusted and logarithmically transformed frequency component L_{ik} can be expressed as:

$$L_{ik} = \frac{\ln(p_{ik} + 1)}{C} \quad (10)$$

$$L_i = [L_{i1}, L_{i2}, \dots, L_{i\frac{N}{2}}] \quad (11)$$

Similarly, given a frequency-domain trace x_i^F , its augmented version $\tilde{x}_i^F$ is obtained using a frequency-based augmentation strategy. Both x_i^F and $\tilde{x}_i^F$ are processed by the contrastive frequency encoder G_F , generating embeddings $h_i^F = G_F(x_i^F)$ and $\tilde{h}_i^F = G_F(\tilde{x}_i^F)$. These embeddings are then passed through the frequency projector R_F , yielding outputs in lower-dimensional space: $z_i^F = R_F(h_i^F)$ and $\tilde{z}_i^F = R_F(\tilde{h}_i^F)$. As $\tilde{x}_i^F$ is the augmented version of x_i^F , they are expected to be closer in the frequency embedding space, while the distance between x_i^F and any other sample x_j^F (or its augmented version $\tilde{x}_j^F$) will be larger. Consequently, the contrastive loss maximizes the similarity of positive pairs, such as $(x_i^F, \tilde{x}_i^F)$, while minimizing the similarity of negative pairs like (x_i^F, x_j^F) and $(x_i^F, \tilde{x}_j^F)$. The frequency-based contrastive loss is calculated as follows:

$$\mathcal{L}_{F,i} = -\log \frac{\exp(\text{sim}(z_i^F, \tilde{z}_i^F) / \tau)}{\sum_{x_j \in D^{\text{pre}} \mathbb{1}_{[i \neq j]} \exp(\text{sim}(z_i^F, R_F(G_F(x_j^F))) / \tau)} \quad (12)$$

4) *Time-Frequency Consistency*: An original trace x_i^T processed through the time encoder G_T and the frequency encoder G_F , followed by projectors R_T and R_F . These components embed it into a latent time-frequency space, producing four embeddings: $z_i^T = R_T(G_T(x_i^T))$, $\tilde{z}_i^T = R_T(G_T(\tilde{x}_i^T))$, $z_i^F = R_F(G_F(x_i^F))$, $\tilde{z}_i^F = R_F(G_F(\tilde{x}_i^F))$. The first two embeddings are derived from time-domain analysis, while the latter two are obtained from frequency-domain analysis. By closely examining these embeddings, z_i^T and $\tilde{z}_i^T$ are derived from the original trace x_i^T and its frequency representation x_i^F , while $\tilde{z}_i^T$ and $\tilde{z}_i^F$ are from their augmented versions $\tilde{x}_i^T$ and $\tilde{x}_i^F$. Intuitively, z_i^T and $\tilde{z}_i^T$ should be more closely aligned in the latent time-frequency space. Here, we do not consider the similarity between $\tilde{z}_i^T$ and $\tilde{z}_i^F$, as it may not be well-preserved after augmentation. This cross-domain alignment enhances the model's ability to learn joint representations and improves generalization in unseen network conditions. Thus, the time-frequency consistency loss is calculated as:

$$\mathcal{L}_{C,i} = -\log \frac{\exp(\text{sim}(z_i^T, \tilde{z}_i^T) / \tau)}{\sum_{x_j \in D^{\text{pre}} \mathbb{1}_{[i \neq j]} \exp(\text{sim}(z_i^T, R_T(G_T(x_j^T))) / \tau)} \quad (13)$$

The overall loss of WF-TFC consists of three components: time-based contrastive loss $\mathcal{L}_{T,i}$ to ensure invariance in the

time domain, frequency-based contrastive loss $\mathcal{L}_{F,i}$ to enforce similarity in the frequency domain, and time-frequency consistency loss $\mathcal{L}_{C,i}$ to align embeddings across both domains. In summary, the pre-training loss is defined as:

$$\mathcal{L}_i = \lambda (\mathcal{L}_{T,i} + \mathcal{L}_{F,i}) + (1 - \lambda) \mathcal{L}_{C,i} \quad (14)$$

Here, λ is a hyperparameter that controls the relative importance of $\mathcal{L}_{T,i}$, $\mathcal{L}_{F,i}$ and $\mathcal{L}_{C,i}$. The total contrastive loss in the pre-training model is obtained by aggregating $\mathcal{L}_i$ across all traces in the dataset, while the individual losses are computed per mini-batch.

E. Task-Specific Fine-Tuning

With the pre-trained task-agnostic model, the next step is to adapt it to specific few-shot tasks and fine-tune it using a limited number of labeled traces from new websites. In this phase, the pre-trained model is modified by replacing its projection layer with two fully connected layers, with the final layer outputting probabilities for each website. The entire network is then fine-tuned, with both the feature extraction layers and the new fully connected layers are optimized.

The whole fine-tuning process involves three key stages: training, validation and testing. Initially, the adversary gathers N traces from each website, with N typically set to 5, 10, 15, and 20. During training, these traces are fed into the pre-trained model's feature extractor to generate embeddings. Validation verifies the effectiveness of the model, tunes hyperparameters and mitigates overfitting, while the testing phase evaluates model's classification performance on unseen traces.

V. EXPERIMENTS

In this section, we design and conduct a series of experiments to evaluate the effectiveness of WF-TFC across various few-shot tasks.

A. Experiment Setup

1) *Datasets and Composition*: We conduct experiments using two benchmark datasets widely used in prior research. Their composition in this work is labeled as follows:

AWF Dataset [8]. The largest dataset, collected in 2016 using TBB version 6.5, includes websites from both closed-world and open-world scenarios. It also incorporates a time drift component for the first 200 websites, with traces re-collected at various intervals. We categorize the AWF dataset into several different sets:

- **AWF100**: The first 100 monitored websites.
- **AWF200**: The first 200 monitored websites.
- **AWF200-AWF100**: Monitored websites included in AWF200 but not in AWF100.
- **AWF200_{gaps}**: The set of AWF200 re-collected after gaps of 3 days, 10 days, 2 weeks, 4 weeks, and 6 weeks.
- **AWF μ K**: A subset of $\mu \times 1000$ unmonitored websites from the total of 400,000.

DS-19 Dataset [27]. The latest dataset, captured in 2019 using Tor Browser 8.5a7, includes 100 traces for each of 100 monitored websites and one trace for each of 1000

unmonitored websites, with packet timestamps and directions. We further implement the WTF-PAD, FRONT, and TAMRAW anti-anonymity defense strategies to generate the corresponding defended datasets:

- $DS-19_{cw}$: The set of monitored websites.
- $DS-19_{ow}$: The set of unmonitored websites.
- $DS-19_{WTF-PAD}$: $DS-19_{cw}$ defended with WTF-PAD.
- $DS-19_{FRONT}$: $DS-19_{cw}$ defended with FRONT.
- $DS-19_{TAMRAW}$: $DS-19_{cw}$ defended with TAMARAW.

2) *Baselines*: This section outlines the existing WF methods used to benchmark WF-TFC, along with brief descriptions of each. Publicly available code for several algorithms is utilized to ensure results fidelity, while others are re-implemented based on their original papers. To maintain consistency, all methods are evaluated using the same datasets and experimental conditions, minimizing external variables and enabling an objective performance comparison.

- **k-fingerprinting(k-FP)** [20]: Utilizes Random Forests for feature transformation and KNN for classification, making it the most effective manual feature-based method.
- **Automated website fingerprinting(AWF)** [8]: Employs deep learning for automatic feature extraction, enhancing resilience to network changes.
- **Deep Fingerprinting(DF)** [9]: Utilizes a specialized CNN architecture, surpassing previous methods and establishing a standard for effective feature extraction in subsequent WF studies. Notably, it follows a fully supervised learning paradigm.
- **WF-Transformer** [22]: Leverages Transformer for temporal feature extraction, achieving superior performance with shorter input lengths.
- **Triplet Fingerprinting(TF)** [35]: Uses a Triplet network for N-shot learning, with a large labeled traffic used for pre-training.
- **Var-CNN** [38]: A semi-automated method that combines automatic feature extraction with handcrafted features, designed for low-data settings.
- **TLFA** [40]: Enhances classification with large labeled traces from non-target websites for few-shot learning.
- **NetCLR** [44]: Improves classification performance by augmenting network traces.

3) *Implementation Details: Data Representation*: Following standard data representation methods from previous WF works, each trace is represented by packet direction: outgoing packets as 1 and incoming packets as -1. Traces are padded or trimmed to a fixed length of 5000, which gives a final input matrix of size $[n \times 5000]$, where n is the number of traces.

Data Partitioning: The model is pre-trained on the AWF100 dataset, with 2500 traces per website, and evaluated across various settings during fine-tuning. In the closed-world setting, classifiers are trained using $N = \{5, 10, 15, 20\}$ traces randomly selected from each website. The remaining traces are randomly divided into a validation set of 20 traces and a test set of 50 traces per website, ensuring mutual exclusivity among the training, validation, and test sets. For the open-world scenario, we use the same pre-trained model. During fine-tuning, the

TABLE I
HYPERPARAMETERS OF WF-TFC

Hyperparameters	Pre-training	Fine-Tuning
Learning rate	5×10^{-4}	5×10^{-4}
Epochs	100	100
Batch size	128	16
Optimizer	Adam with Cosine Scheduler	Adam
Embedding size	512	512
Output size	128	Number of websites
Dropout	-	0.9
$r_{inc,in}$	1.0	-
$r_{inc,out}$	1.0	-
$r_{dec,in}$	0.5	-
$r_{dec,out}$	0.5	-
th_{in}	10	-
th_{out}	2	-
r_{add}	0.1	-
r_{remove}	0.1	-
λ	0.8	-
τ	0.5	-

training and validation set are balanced with equal size of monitored and unmonitored traces. The test set includes more traces than in the closed-world scenario, providing a more realistic evaluation of the model's performance. We conduct five random tests, reporting the mean and standard deviation(%) with the best result in bold. In the fine-tuning phase, datasets are randomly shuffled for each iteration to ensure variability in the training, validation, and test sets, preventing bias and enhancing the generalizability of our findings.

Model Hyperparameters: The hyperparameters for WF-TFC, including those for trace augmentation, are listed in Table I. Each is selected from a range of candidates, with the final choices reflecting the best relative performance.

Experimental Environment. All experiments are conducted on a multi-core server with a 20-core 2.20 GHz Intel Xeon CPU and an NVIDIA Tesla V100 GPU, providing the computational power necessary for efficient training and evaluation of WF-TFC.

Evaluation Metrics. In closed-world evaluation, accuracy quantifies the proportion of correctly classified monitored websites, highlighting the model's effectiveness in distinguishing and classifying traffic across multiple websites. For the open-world scenario, we employ F1 score as key metric, particularly due to the presence of numerous unmonitored websites. As the harmonic mean of precision and recall, the F1 score balances the trade-off between these two metrics. Precision measures the accuracy of predicting monitored websites, while recall evaluates the model's ability to identify all monitored sites. By combining both metrics, it offers a balanced assessment of the model's effectiveness in distinguishing between monitored and unmonitored websites.

B. Closed-World Scenarios

This section presents the experimental results of WF-TFC in closed-world scenarios, evaluating its performance with pre-training and fine-tuning datasets from both similar and drifted distributions. We also assess its robustness to concept drift and different anti-anonymity defenses.

TABLE II

CLOSED-WORLD: COMPARISON WITH BASELINE USING SIMILAR BUT MUTUALLY EXCLUSIVE DATASET

Methods	5-shot	10-shot	15-shot	20-shot
K-FP	77.72 $\pm$ 1.60	84.16 $\pm$ 1.10	86.81 $\pm$ 0.62	87.16 $\pm$ 0.75
AWF	34.32 $\pm$ 1.48	43.00 $\pm$ 2.42	55.04 $\pm$ 3.00	62.09 $\pm$ 0.99
DF	74.98 $\pm$ 1.60	85.46 $\pm$ 2.92	90.93 $\pm$ 0.87	92.94 $\pm$ 0.96
WT-Transformer	31.53 $\pm$ 0.90	40.62 $\pm$ 0.55	48.94 $\pm$ 1.44	57.73 $\pm$ 0.68
TF	80.96 $\pm$ 0.68	85.42 $\pm$ 0.35	87.28 $\pm$ 0.36	87.94 $\pm$ 0.23
Var-CNN	31.42 $\pm$ 1.84	36.20 $\pm$ 1.71	41.67 $\pm$ 2.09	47.68 $\pm$ 1.29
TLFA	88.38 $\pm$ 0.58	91.86 $\pm$ 0.49	93.03 $\pm$ 0.52	93.63 $\pm$ 0.83
NetCLR	90.45 $\pm$ 0.60	95.43 $\pm$ 0.38	97.13 $\pm$ 0.17	97.42 $\pm$ 0.23
WF-TFC	95.02 $\pm$ 0.41	97.10 $\pm$ 0.21	97.74 $\pm$ 0.14	98.02 $\pm$ 0.14

TABLE III

CLOSED-WORLD: COMPARISON UNDER DIFFERENT DISTRIBUTION DATASET

Methods	5-shot	10-shot	15-shot	20-shot
DF	74.03 $\pm$ 2.54	87.14 $\pm$ 0.99	90.54 $\pm$ 1.39	93.00 $\pm$ 0.95
TF	84.34 $\pm$ 0.55	88.35 $\pm$ 0.42	89.98 $\pm$ 0.32	91.12 $\pm$ 0.31
TLFA	75.48 $\pm$ 0.47	81.51 $\pm$ 1.08	83.86 $\pm$ 0.47	85.36 $\pm$ 0.52
NetCLR	87.57 $\pm$ 0.87	92.45 $\pm$ 0.48	93.75 $\pm$ 0.49	94.88 $\pm$ 0.32
WF-TFC	90.77 $\pm$ 0.85	94.38 $\pm$ 0.46	95.20 $\pm$ 0.35	96.06 $\pm$ 0.10

1) *Similar but Mutually Exclusive Dataset*: In this setting, the adversary uses traces collected under the same conditions for both pre-training and fine-tuning. Here, we utilize the AWF200-AWF100 dataset for classification. Table II compares WF-TFC with baselines across different N values (number of traces per website) in few-shot scenarios.

As shown in Table II, the accuracy of all methods improves with an increasing of N. WF-TFC consistently outperforms baselines across all N values, particularly when N is smaller. For instance, with 5-shot learning, WF-TFC achieves an accuracy of 95.02%, compared to 90.45% for the SOTA (i.e., NetCLR). In contrast, other methods perform poorly, with AWF, WT-Transformer, and Var-CNN even falling below 35% accuracy. This indicates that WF-TFC enhances the model's generalization ability with fewer labeled traces.

2) *Different Distribution Dataset*: Next, WF-TFC's performance is evaluated in a realistic setting, using pre-training and classification datasets from different distributions, collected at various times, with different TBB versions, and under changing network conditions. For fine-tuning, we employ the DS-19_{cw} dataset, collected three years after the pre-training dataset with updated TBBs. This setup assesses WF-TFC's robustness and generalization with data collected at different periods, reflecting real-world scenarios with evolving browser technologies and network conditions.

Table III shows the classification results with various values of N, compared to the top-performing baselines from previous experiments. Although accuracy decreases compared to the same distribution, WF-TFC still achieves the highest accuracy. As N increases, the gap narrows, but WF-TFC consistently outperforms the other methods. With 20-shot learning, WF-TFC achieves an accuracy of 96.06%, exceeding 94.88% of SOTA (i.e., NetCLR) and 93.00% of the superior fully supervised DF model. This shows that WF-TFC improves the knowledge transferability from the pre-trained model.

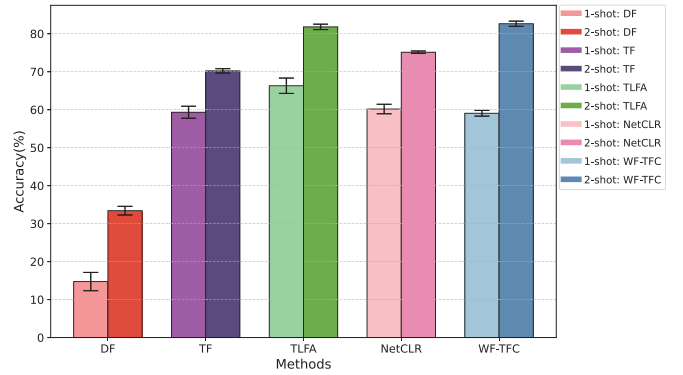


Fig. 5. Closed-world: comparison with error bars in challenging few-shot scenarios: 1-shot (light colors) and 2-shot (dark colors).

3) *Against Concept Drift*: In this part, we evaluate the robustness of WF-TFC against concept drift. The model is initially pre-trained on the AWF200 dataset and later evaluated on AWF200_{gaps}. This setup assesses the model's adaptability, particularly its ability to recognize previously learned websites in new, unseen traces over time. It is crucial for real-world scenarios, where the evolving characteristics of network traces can significantly affect performance. Table IV gives the comparison results.

As expected, accuracy improves for all methods with an increase in N, but decreases as the time interval between test and pre-training data increases. WF-TFC exhibits the slowest decline, demonstrating the strongest resistance to concept drift. For instance, in 5-shot learning, it drops by only 3.24% from 3 days to 6 weeks, compared to a 4.20% decline for the SOTA (i.e., NetCLR). Although TLFA, which relies on a large number of labeled traces during pre-training, performs best with a 3-day interval, its accuracy declines rapidly as the interval increases, falling below that of WF-TFC. Additionally, WF-TFC outperforms the fully supervised DF by more than 10% over a 6-week interval, even with 20-shot learning. Overall, WF-TFC enhances data compatibility and knowledge transferability between learned and unseen websites, performing optimally with longer drift intervals or smaller N, making it well-suited for few-shot scenarios and highly resistant to concept drift.

In order to evaluate the performance of WF-TFC under more challenging settings, we perform 1-shot and 2-shot classification with a time gap of over 6 weeks between the pre-training and test datasets. Figure 5 shows the results and compares them with the baselines.

As can be observed, as the number of samples per website increases, the accuracy of all methods improves, with WF-TFC showing the largest improvement. With 1-shot learning, TLFA achieves the highest performance, benefiting from extensive labeled traffic pre-training. Notably, WF-TFC can rapidly increase accuracy when trained with two traces per website and achieves the best result. In 1-shot learning, the extreme scarcity of data samples means that a single sample may not capture sufficient temporal information, resulting in limited time-domain features. The contribution of frequency-domain features is also minimal in this setting. However, as the number

TABLE IV
COMPARISON AGAINST CONCEPT DRIFT

N-shot	methods	3days	10days	2weeks	4weeks	6weeks
5	DF	67.25 $\pm$ 2.21	67.13 $\pm$ 1.35	66.06 $\pm$ 1.43	63.24 $\pm$ 2.29	61.70 $\pm$ 2.67
	TF	85.34 $\pm$ 0.81	84.97 $\pm$ 0.52	83.61 $\pm$ 0.70	81.22 $\pm$ 0.39	77.96 $\pm$ 0.39
	TLFA	95.91 $\pm$ 0.41	95.17 $\pm$ 0.31	94.27 $\pm$ 0.34	91.65 $\pm$ 0.53	90.50 $\pm$ 0.44
	NetCLR	93.31 $\pm$ 0.33	92.79 $\pm$ 0.43	92.71 $\pm$ 0.53	90.32 $\pm$ 0.41	89.11 $\pm$ 0.43
	WF-TFC	95.86 $\pm$ 0.53	95.56 $\pm$ 0.27	95.60 $\pm$ 0.42	93.22 $\pm$ 0.33	92.62 $\pm$ 0.43
10	DF	81.68 $\pm$ 1.53	82.12 $\pm$ 0.53	79.80 $\pm$ 1.31	76.49 $\pm$ 1.88	73.50 $\pm$ 2.90
	TF	88.50 $\pm$ 0.40	88.76 $\pm$ 0.26	87.34 $\pm$ 0.20	84.97 $\pm$ 0.48	82.14 $\pm$ 0.49
	TLFA	97.31 $\pm$ 0.33	96.96 $\pm$ 0.25	96.39 $\pm$ 0.20	93.81 $\pm$ 0.24	93.31 $\pm$ 0.27
	NetCLR	96.55 $\pm$ 0.11	96.62 $\pm$ 0.17	96.34 $\pm$ 0.24	94.36 $\pm$ 0.38	93.96 $\pm$ 0.35
	WF-TFC	97.10 $\pm$ 0.16	97.02 $\pm$ 0.16	96.86 $\pm$ 0.29	95.05 $\pm$ 0.53	94.87 $\pm$ 0.24
15	DF	86.77 $\pm$ 1.53	88.30 $\pm$ 0.38	86.36 $\pm$ 1.08	84.69 $\pm$ 0.64	81.06 $\pm$ 1.27
	TF	89.90 $\pm$ 0.62	90.16 $\pm$ 0.36	88.97 $\pm$ 0.42	86.53 $\pm$ 0.35	84.03 $\pm$ 0.44
	TLFA	98.02 $\pm$ 0.11	97.48 $\pm$ 0.15	96.88 $\pm$ 0.08	94.88 $\pm$ 0.18	94.04 $\pm$ 0.36
	NetCLR	97.57 $\pm$ 0.16	97.35 $\pm$ 0.20	97.24 $\pm$ 0.08	95.68 $\pm$ 0.13	95.41 $\pm$ 0.22
	WF-TFC	97.67 $\pm$ 0.17	97.58 $\pm$ 0.16	97.44 $\pm$ 0.21	95.93 $\pm$ 0.06	95.53 $\pm$ 0.18
20	DF	89.30 $\pm$ 0.46	88.92 $\pm$ 0.52	88.02 $\pm$ 0.95	86.07 $\pm$ 0.82	85.68 $\pm$ 1.00
	TF	90.95 $\pm$ 0.43	91.17 $\pm$ 0.27	90.06 $\pm$ 0.15	87.70 $\pm$ 0.23	85.24 $\pm$ 0.25
	TLFA	98.15 $\pm$ 0.15	97.77 $\pm$ 0.27	97.32 $\pm$ 0.22	95.18 $\pm$ 0.25	94.48 $\pm$ 0.24
	NetCLR	98.06 $\pm$ 0.15	97.72 $\pm$ 0.14	97.61 $\pm$ 0.17	96.23 $\pm$ 0.11	96.11 $\pm$ 0.21
	WF-TFC	97.93 $\pm$ 0.16	97.79 $\pm$ 0.20	97.67 $\pm$ 0.17	96.41 $\pm$ 0.10	96.17 $\pm$ 0.20

TABLE V
COMPARISON UNDER DEFENDED DATASETS

Defenses	N-shot	DF	TF	TLFA	NetCLR	WF-TFC
WTF-PAD	5	49.62 $\pm$ 1.52	26.01 $\pm$ 0.90	32.99 $\pm$ 0.51	55.87 $\pm$ 0.76	65.62 $\pm$ 1.36
	10	67.88 $\pm$ 1.09	31.32 $\pm$ 0.17	40.99 $\pm$ 0.65	70.40 $\pm$ 0.77	81.06 $\pm$ 0.61
	15	76.91 $\pm$ 1.60	34.52 $\pm$ 0.78	43.99 $\pm$ 0.69	76.98 $\pm$ 0.43	86.06 $\pm$ 0.19
	20	81.47 $\pm$ 1.06	36.05 $\pm$ 0.87	45.81 $\pm$ 1.36	81.22 $\pm$ 0.61	88.72 $\pm$ 0.40
FRONT	5	6.65 $\pm$ 1.50	3.71 $\pm$ 0.38	1.78 $\pm$ 0.15	6.92 $\pm$ 0.43	8.23 $\pm$ 0.44
	10	13.66 $\pm$ 1.53	4.69 $\pm$ 0.39	1.82 $\pm$ 0.18	10.30 $\pm$ 0.52	17.76 $\pm$ 1.08
	15	19.85 $\pm$ 2.34	5.23 $\pm$ 0.37	2.01 $\pm$ 0.11	14.33 $\pm$ 0.28	32.46 $\pm$ 1.40
	20	23.40 $\pm$ 3.33	5.79 $\pm$ 0.42	1.86 $\pm$ 0.23	18.53 $\pm$ 0.61	42.01 $\pm$ 1.33
TAMARAW	5	6.00 $\pm$ 0.79	6.37 $\pm$ 0.45	6.07 $\pm$ 0.56	7.44 $\pm$ 0.71	7.36 $\pm$ 0.57
	10	7.40 $\pm$ 0.97	6.99 $\pm$ 0.42	6.40 $\pm$ 0.47	9.53 $\pm$ 0.55	9.64 $\pm$ 0.79
	15	7.40 $\pm$ 0.36	6.82 $\pm$ 0.30	6.13 $\pm$ 0.35	9.93 $\pm$ 0.54	10.49 $\pm$ 0.35
	20	7.59 $\pm$ 1.04	7.50 $\pm$ 0.54	5.92 $\pm$ 0.32	10.76 $\pm$ 0.42	11.33 $\pm$ 0.48

of traces increases to two per website, the advantages of frequency-domain features become more apparent. They effectively complement the shortcomings of time-domain features, enabling the model to achieve more accurate classification performance.

4) *Against Anti-Anonymity Defenses:* The more challenging few-shot setting is further explored on traces using various anti-anonymity defenses. We utilize the DS-19_{WTF-PAD}, DS-19_{FRONT} and DS-19_{TAMARAW} datasets to assess WF-TFC's transferability to countermeasures, and the results are presented in Table V.

As is shown, all three anti-anonymity defenses effectively mitigate existing methods, leading to a notable decline in accuracy compared to undefended traces. WTF-PAD shows the weakest performance, while the heavyweight TAMARAW exhibits the strongest defense effectiveness. As N increases, the accuracy of all methods improves, and WF-TFC exceeds all other methods in nearly all settings. With 10-shot learning under the WTF-PAD defense, WF-TFC achieves 81.06% accuracy, exceeding the SOTA(i.e., NetCLR) by over 10%.

Although TAMARAW provides strong defense, its high bandwidth overhead and latency limit practicality, emphasizing the need to balance defense and efficiency.

C. Open-World Scenarios

The following experiments investigate a more realistic and challenging open-world scenario, where the adversary must differentiate between a limited set of monitored websites and a large number of unmonitored ones. The pre-trained model is fine-tuned with a standard model that incorporates unmonitored websites, exposing it to a broader range of browsing behaviors and evaluating its real-world performance.

1) *Similar but Mutually Exclusive Dataset:* This part assesses the performance of WF-TFC in a setting where the pre-training and classification datasets share a similar distribution but remain mutually exclusive. By ensuring no overlap, the model's ability to generalize to unseen data is rigorously assessed. Here, we use the AWF200-AWF100 dataset for monitored websites and the AWF10K dataset for

TABLE VI
OPEN-WORLD: COMPARISON ON SIMILAR BUT MUTUALLY EXCLUSIVE DATASET

Methods	Turned for precision				Turned for recall			
	5-shot	10-shot	15-shot	20-shot	5-shot	10-shot	15-shot	20-shot
DF	63.32 $\pm$ 5.75	81.97 $\pm$ 0.60	85.58 $\pm$ 1.73	89.23 $\pm$ 0.28	74.80 $\pm$ 0.66	82.46 $\pm$ 0.92	85.51 $\pm$ 0.52	88.16 $\pm$ 0.95
TF	66.84 $\pm$ 0.58	66.09 $\pm$ 1.80	68.44 $\pm$ 2.33	69.29 $\pm$ 3.89	66.84 $\pm$ 0.58	71.80 $\pm$ 1.60	74.43 $\pm$ 0.37	75.52 $\pm$ 0.74
TLFA	28.74 $\pm$ 2.32	49.89 $\pm$ 0.76	56.85 $\pm$ 1.25	62.12 $\pm$ 2.11	73.79 $\pm$ 0.33	79.87 $\pm$ 0.47	81.71 $\pm$ 0.30	82.86 $\pm$ 0.54
NetCLR	36.43 $\pm$ 2.35	74.38 $\pm$ 1.51	83.61 $\pm$ 2.09	87.99 $\pm$ 0.85	81.08 $\pm$ 0.85	90.10 $\pm$ 0.50	92.15 $\pm$ 0.37	93.22 $\pm$ 0.31
WF-TFC	81.54 $\pm$ 1.85	89.77 $\pm$ 1.13	91.09 $\pm$ 1.07	92.98 $\pm$ 0.35	87.20 $\pm$ 0.75	91.23 $\pm$ 0.28	92.53 $\pm$ 0.74	93.26 $\pm$ 0.11

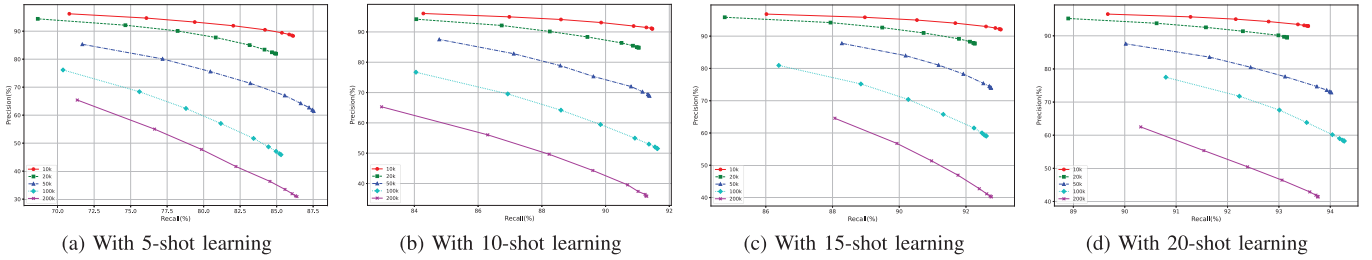


Fig. 6. Open-world: precision-recall curves for increasing unmonitored websites.

unmonitored websites. Table VI shows the comparison results for different values of N , with models turned for precision and recall, respectively.

Regardless of whether the models are turned for precision or recall, WF-TFC consistently achieves the highest F1 scores across all N values, with its performance advantage being more pronounced in precision-turned models. For example, in the 10-shot learning with models turned for precision, WF-TFC achieves an F1 score of 89.77%, outperforming the well-performing DF and NetCLR by 7.80% and 15.39%, respectively. When models are tuned for recall, WF-TFC obtains an F1 score of 91.23%, compared to 82.46% for DF and 90.10% for NetCLR. This demonstrates that WF-TFC strikes an effective balance between precision and recall, with its advantage being more evident at smaller N . As N increases, the performance gap between WF-TFC and the baselines narrows, highlighting its strong adaptability to fewer traffic.

2) *In Larger Open-World*: As the number of open-world websites increases, it becomes essential to assess the effectiveness of WF-TFC on a larger and more diverse set of unmonitored websites. We aim to evaluate its performance with varying unmonitored traces, enhancing our understanding of its scalability and robustness in dynamic network environments. Figure 6 shows the precision-recall curves for different N across unmonitored website sets: AWF10k, AWF20k, AWF50k, AWF100k, and AWF200k.

When the number of unmonitored websites is fixed, increasing N shifts the precision-recall curves toward the top right, indicating improved performance. This reflects the model's enhanced ability to learn features and differentiate websites more effectively by utilizing additional training samples, thereby strengthening its generalization. Conversely, when N is constant, an increase in unmonitored websites causes a downward shift in the curves, signaling a decline in

precision and recall. This decline likely arises from increased interference and noise from the diverse unmonitored websites, challenging the model's capability to accurately distinguish target websites.

With 200,000 unmonitored websites in 10-shot learning, 40 times the number of monitored websites (Figure 6(b)), WF-TFC achieves a precision of 65.30% and a recall of 82.97% when the models are tuned for precision. Tuning for recall results in 35.79% precision and 91.29% recall, highlighting a trade-off where the model identifies more monitored websites at the cost of precision. As expected, performance improves as the number of unmonitored websites decreases. Given 50,000 unmonitored websites, still 10 times the number of monitored websites, WF-TFC achieves a precision of 87.52% and a recall of 84.78% when models are turned for precision. When tuned for recall, it maintains 68.90% precision and 91.38% recall. This illustrates WF-TFC's robust knowledge transferability while effectively balancing precision and recall.

3) *Distributional Shift Dataset*: Next, WF-TFC is evaluated in a more challenging open-world scenario using a classification dataset that significantly differs from the pre-training data. This includes a three-year time gap and a different version of the TBB, causing substantial distributional shifts, which increases the complexity and provides a robust assessment of WF-TFC's knowledge transferability under real-world conditions. Specifically, we use DS-19_{cw} for monitored websites and DS-19_{ow} for unmonitored websites during fine-tuning and testing. Table VII gives the comparison results for models turned for precision and recall under this setting.

Across all N values, WF-TFC achieves the highest F1 score when models are turned for either precision or recall, surpassing all baselines. Its advantage is particularly pronounced in precision-turned models and smaller N . With only 5 traces per website and models are turned for precision, WF-TFC obtains an F1 score of 84.62%, 2.19% higher than the best

TABLE VII
OPEN-WORLD: COMPARISON UNDER DISTRIBUTIONAL SHIFT DATASET

Methods	Turned for precision				Turned for recall			
	5-shot	10-shot	15-shot	20-shot	5-shot	10-shot	15-shot	20-shot
DF	80.13 $\pm$ 1.78	89.85 $\pm$ 0.42	93.24 $\pm$ 0.59	94.37 $\pm$ 0.98	96.77 $\pm$ 0.10	97.75 $\pm$ 0.06	97.99 $\pm$ 0.09	98.14 $\pm$ 0.14
TF	82.43 $\pm$ 0.18	83.02 $\pm$ 0.35	85.69 $\pm$ 0.61	86.50 $\pm$ 0.34	82.43 $\pm$ 0.18	85.64 $\pm$ 1.52	87.72 $\pm$ 0.57	88.91 $\pm$ 0.16
TLFA	1.23 $\pm$ 0.30	8.54 $\pm$ 0.45	14.70 $\pm$ 0.40	20.99 $\pm$ 1.08	41.88 $\pm$ 1.85	60.18 $\pm$ 1.02	65.78 $\pm$ 0.55	71.10 $\pm$ 1.10
NetCLR	46.32 $\pm$ 2.28	77.07 $\pm$ 1.64	87.67 $\pm$ 0.39	89.94 $\pm$ 0.28	90.64 $\pm$ 0.76	96.24 $\pm$ 0.14	97.23 $\pm$ 0.15	97.59 $\pm$ 0.17
WF-TFC	84.62 $\pm$ 1.95	92.12 $\pm$ 0.39	94.44 $\pm$ 0.72	94.69 $\pm$ 0.47	96.81 $\pm$ 0.15	97.80 $\pm$ 0.18	98.09 $\pm$ 0.08	98.28 $\pm$ 0.11

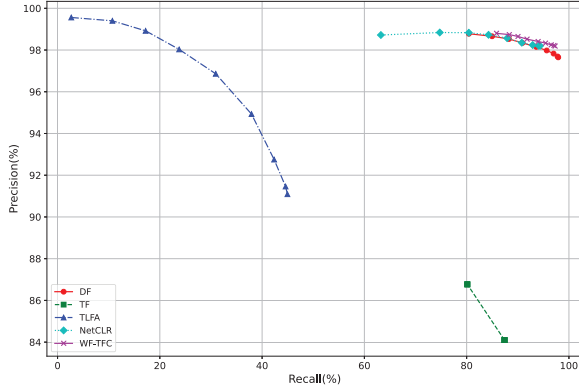


Fig. 7. Open-world: precision-recall curves on distributional shift dataset for 10-shot learning.

result of TF. In contrast, TLFA performs the poorest with an F1 score not exceeding 25%, indicating that direct transfer learning is largely ineffective in this challenging scenario. When the models are turned for recall, NetCLR achieves an impressive F1 score of 90.64%, but still 6.17% lower than that of WF-TFC. Meanwhile, the fully supervised DF model also performs well, and TLFA consistently exhibits the lowest F1 score. In summary, WF-TFC enhances data compatibility under distributional shift settings and effectively identifies monitored websites while minimizing the misclassification of unmonitored websites with fewer traces.

Additionally, Figure 7 shows precision-recall curves for 10-shot learning, comparing different methods over a range of thresholds. It can be observed that WF-TFC strikes the best balance between precision and recall, particularly at higher recall levels. DF and NetCLR maintain high precision but demonstrate a trade-off as recall increases. In contrast, TF and TLFA show high precision but struggle with low recall, limiting their effectiveness in distinguishing monitored websites.

To further investigate the effectiveness of WF-TFC, we conduct comparison tests under extreme 1-shot and 2-shot settings, as shown in Figure 8. In 1-shot learning, WF-TFC achieves the highest F1 score, followed by DF but with highest variance, while NetCLR's F1 score drops to 0.00% due to insufficient samples. All methods show improved classification performance when two traces from each website are used for fine-tuning. To be specific, WF-TFC obtains an F1 score of 92.72%, surpassing the best-performing few-shot method (i.e., TF) by over 13%. However, it is exceeded by the fully

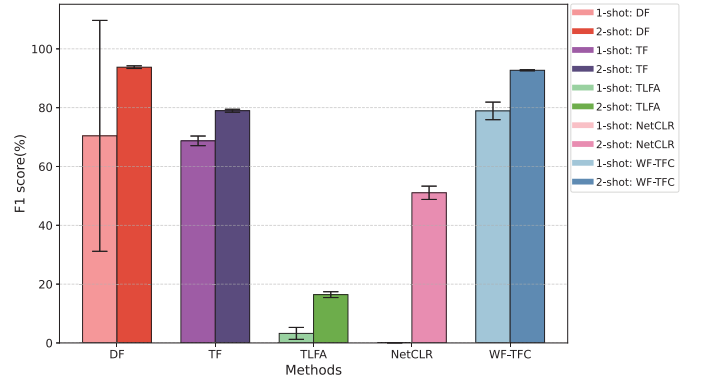


Fig. 8. Open-world: comparison with error bars under challenging few-shot settings: 1-shot (light colors) and 2-shot (dark colors).

TABLE VIII
CLASSIFICATION ACCURACY FOR ABLATION STUDY

Model	5-shot	10-shot	15-shot	20-shot
W/o $\mathcal{L}_F$ & $\mathcal{L}_C$	70.61 $\pm$ 2.74	91.63 $\pm$ 0.59	94.30 $\pm$ 0.41	96.01 $\pm$ 0.39
W/o $\mathcal{L}_T$ & $\mathcal{L}_C$	43.99 $\pm$ 1.98	53.02 $\pm$ 1.30	58.08 $\pm$ 0.65	60.87 $\pm$ 1.16
W/o $\mathcal{L}_C$	73.84 $\pm$ 0.34	93.36 $\pm$ 0.77	95.71 $\pm$ 0.46	96.48 $\pm$ 0.54
W/o Aug_{in}	93.00 $\pm$ 0.75	96.31 $\pm$ 0.24	97.14 $\pm$ 0.17	97.51 $\pm$ 0.23
W/o Aug_{out}	91.00 $\pm$ 0.45	95.13 $\pm$ 0.39	96.38 $\pm$ 0.32	96.85 $\pm$ 0.22
WF-TFC	95.02 $\pm$ 0.41	97.10 $\pm$ 0.21	97.74 $\pm$ 0.14	98.02 $\pm$ 0.14

supervised DF model by 1.05%, with WF-TFC showing lower variance. Overall, WF-TFC exhibits better transferability and robustness in extreme few-shot scenarios.

D. Ablation Study

This section evaluates the contribution of each component in WF-TFC by systematically removing them and assessing their impact on performance. As an example in the closed-world scenario, Table VIII gives the results for the model fine-tuned on the AWF200-AWF100 dataset. If $\mathcal{L}_F$ is removed, $\mathcal{L}_C$ cannot be calculated, so both $\mathcal{L}_F$ and $\mathcal{L}_C$ are removed. The same applies to $\mathcal{L}_T$. “W/o $\mathcal{L}_F$ & $\mathcal{L}_C$ ” means excluding both the frequency contrastive encoder and the time-frequency consistency module, while “W/o $\mathcal{L}_T$ & $\mathcal{L}_C$ ” indicates that only the frequency contrastive encoder is included. “W/o $\mathcal{L}_C$ ” means removing the time-frequency consistency module and using $[z^T, z^F]$ as embeddings for downstream tasks. Meanwhile, we separately present the distinct effects of augmenting the incoming and outgoing flows. “W/o Aug_{in} ” indicates that

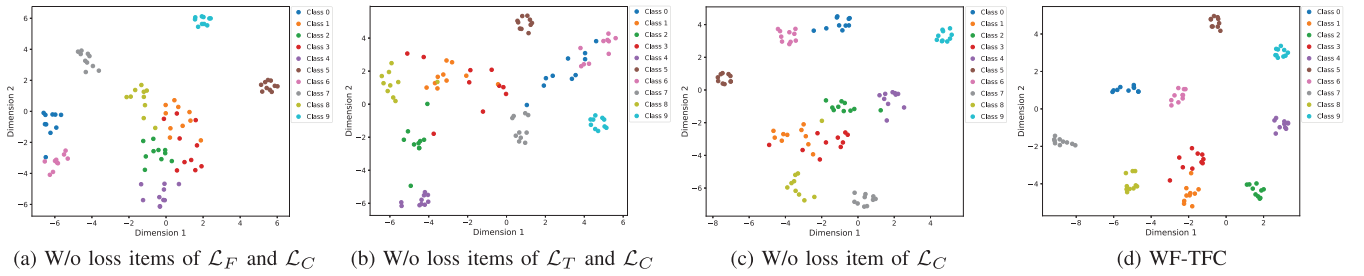


Fig. 9. Open-world: visualizing of embedding using t-SNE.

only the outgoing flows are augmented, while “W/o Aug_{out} ” means augmentation is applied solely to the incoming flows.

The results indicate that time-based pre-training achieves higher accuracy than the frequency-based method. Excluding the time-frequency consistency module, incorporating frequency information still improves the model’s performance. With the full model that includes the time-frequency consistency module, accuracy improves further. For instance, with 10-shot learning, WF-TFC achieves 97.10% accuracy, followed by the other models at 93.36%, 91.63%, and 53.02%, respectively. Regarding augmentation methods, enhancing the outgoing flow is more effective than the incoming flow, as it carries more user behavior-related information and is less influenced by external interference, resulting in improved accuracy. Consequently, WF-TFC achieves the best classification performance.

Furthermore, to intuitively evaluate the effectiveness of WF-TFC in leveraging time and frequency information, along with their consistency during pre-training, we visualize the learned embeddings in time-frequency space. The pre-trained model is fine-tuned by randomly selecting 10 websites from the AWF200-AWF100 dataset and then randomly choosing 10 traces from each for testing. The learned embeddings of these test samples are visualized using t-SNE [53], as shown in Figure 9.

It is evident that the separate time- and frequency-based encoders achieve effective clustering, although traffic from some websites remain dispersed. Combining their embeddings directly still improve cohesion, bringing traces from the same website closer together. Incorporating time-frequency consistency further amplifies this effect by aligning time- and frequency-based features, enhancing cohesion among traces from the same website while increasing separation from those of different websites. This demonstrates the effectiveness of WF-TFC in utilizing time and frequency information, as well as their consistency, thereby enhancing data compatibility and knowledge transferability across websites.

VI. DISCUSSION

This paper presents WF-TFC, which integrates self-supervised learning with time-frequency consistency during pre-training, achieving superior performance across various downstream few-shot tasks. However, the ablation study results indicate that time domain analysis alone achieves an accuracy of 91.63% with 10-shot learning, whereas frequency analysis only reaches 53.02%. This discrepancy arises

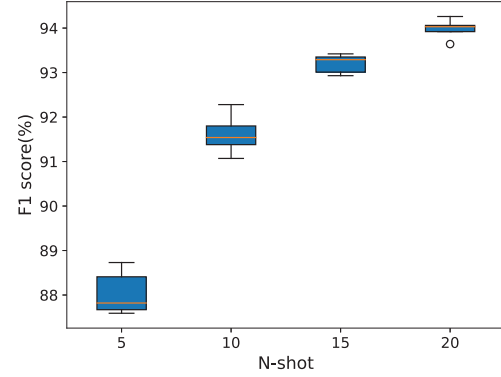


Fig. 10. Open-world: classification performance for different N-shot settings in larger-scale traffic.

from time domain analysis’s ability to capture subtle traffic variations over specific periods, such as packet sending frequency, timing, and latency. These local features relate closely to specific website activity patterns, enabling adversaries to accurately identify traffic characteristics. For instance, some websites may exhibit unique packet sending patterns during peak periods, making them easier to differentiate. While frequency domain analysis reveals global traffic characteristics like periodic patterns and transmission rates, it often misses instantaneous changes and local details, resulting in suboptimal performance. When combined, it becomes more effective, as the overall patterns and periodic features from frequency domain analysis enhance and complement the fine-grained details captured in the time domain.

It is also observed that WF-TFC performance declines on defended traces, but it is still better than the baselines. In the trial deployment of WF-PAD in the Tor browser with 10-shot learning, WF-TFC achieved 81.06% accuracy, while on traces defended by the heavier TAMARAW, it dropped to 9.64%. While TAMARAW effectively counters existing methods, it introduces significant bandwidth overhead and latency, disrupting normal access and making it impractical. Another possible reason for decreased accuracy is that the model is pre-trained on undefended traces but classified on defended ones, with too few traces per website for effective optimization.

Next, we assess the scalability of WF-TFC, specifically examining its performance when dealing with larger-scale network traffic or a greater number of websites in open-world scenarios. To achieve this, we use the AWF200-AWF100

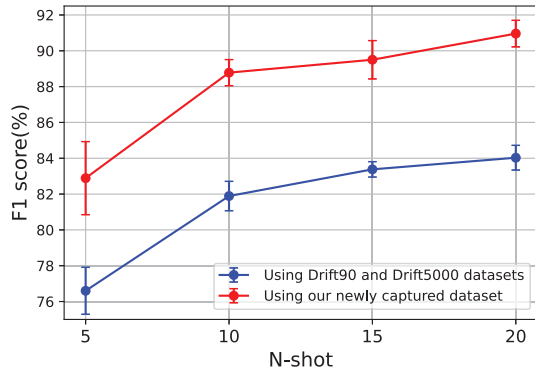


Fig. 11. Open-world: classification results for different N-shot settings across two real-world datasets.

dataset for monitored websites, with all 2,500 traces per website, and the entire AWF400K dataset for unmonitored websites. Figure 10 shows the boxplots for different values of N , with the model turned for precision and recall, respectively. The F1 score increases with N , showing that a larger number of training samples enhance classification performance. Meanwhile, the variance decreases, as indicated by a wider box plot at lower N and a narrower one at higher N , reflecting improved model stability and generalization.

Furthermore, we explore the effectiveness of WF-TFC in realistic open-world settings by incorporating the latest public Drift90 and Drift5000 datasets [44], as well as our newly captured traces. These datasets are collected more than five and eight years after the AWF dataset, on which the WF model is pre-trained. It is worth noting that these datasets better reflect the real-world deployment of the WF model, the Drift90 and Drift5000 datasets collected traces under varying network conditions, including stable connections (superior traces) and poor, low-bandwidth environments (inferior traces). Our newly captured dataset is obtained through real, normal website access using only one client. Since the Tor network has over 2 million daily users, the impact of this single client on the network is expected to be minimal. Specifically, the Drift90 and Drift5000 datasets consist of 93 monitored websites, each with at least 20 inferior and 100 superior traces, along with 5000 unmonitored websites, each having one trace. Meanwhile, our new dataset captured from Alexa-ranked websites, comprising 20 monitored websites with at least 60 traces each and 1600 unmonitored websites with one trace each. Figure 11 presents the average results with error bars for different values of N across these two datasets, based on five random tests.

Similarly, the F1 score improves with the increase of N . Using our newly captured traces, WF-TFC achieves superior performance than Drift90 and Drift5000, reaching an 88.78% F1 score with 10-shot learning, while still maintaining 81.89% on Drift90 and Drift5000 datasets. It is notable that the number of unmonitored traces in the test set is twice that of the monitored ones. We also observe a performance degradation when deploying the WF model on the two more realistic traces compared to its performance on the AWF and DS-19 datasets. This decline is likely due to the significant gap between these traces and the pre-training dataset, along with the increased

variability and complexity in real-world traffic. In the future, we will focus on addressing the challenges of more realistic WF identification with limited labeled traffic, enhancing model robustness in dynamic and evolving networks.

VII. CONCLUSION

In this paper, we introduce WF-TFC, a novel approach tailored for open-world few-shot anonymous WF that utilize self-supervised contrastive pre-training, along with time-frequency consistency. By incorporating both time- and frequency-domain perspectives, traces are augmented to simulate variations in dynamic website content and diverse network environments, enhancing the model's ability to accommodate diverse downstream few-shot tasks. Furthermore, integrating time-frequency consistency during pre-training aligns time- and frequency-based representations within a latent time-frequency space, enabling the model to capture inherent patterns across different websites. As expected, it accommodates various task-specific few-shot scenarios effectively without requiring any labeled or target data during pre-training. Extensive experiments in both closed- and open-world challenging settings demonstrate the effectiveness and superiority of WF-TFC over other methods. Overall, WF-TFC effectively adapts diverse few-shot target domains with varying dynamics, facilitating the model's knowledge transferability and data compatibility while maintaining high classification performance in unobserved long-term temporal and spatial network environments. It also shows potential to support cybersecurity by enhancing behavior localization for anonymous website traffic with limited traces, enabling threat perception and network surveillance in open-world scenarios.

REFERENCES

- [1] R. Dingledine, N. Mathewson, and P. F. Syverson, "Tor: The second-generation onion router," in *Proc. USENIX Secur. Symp.*, Jan. 2004, pp. 21–30.
- [2] (2023). *Tor Metric Portal*. [Online]. Available: <https://metric.torproject.org/>
- [3] I. Karunanayake, N. Ahmed, R. A. Malaney, R. M. D. Islam, and S. K. Jha, "Anonymity with tor: A survey on tor attacks," 2020, *ArXiv: 2009.13018*.
- [4] M. Chawki, "The dark Web and the future of illicit drug markets," *J. Transp. Secur.*, vol. 15, nos. 3–4, pp. 173–191, Dec. 2022.
- [5] P. Dodia, M. AlSabah, O. Alrawi, and T. Wang, "Exposing the rat in the tunnel: Using traffic analysis for tor-based malware detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 875–889.
- [6] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in *Proc. 10th Annu. ACM workshop Privacy Electron. Soc.*, Oct. 2011, pp. 103–114.
- [7] T. Wang and I. Goldberg, "Improved website fingerprinting on tor," in *Proc. 12th ACM workshop Workshop privacy Electron. Soc.*, Nov. 2013, pp. 1–12.
- [8] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," 2017, *arXiv:1708.06376*.
- [9] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 1928–1943.
- [10] Q. Tan, X. Wang, W. Shi, J. Tang, and Z. Tian, "An anonymity vulnerability in tor," *IEEE/ACM Trans. Netw.*, vol. 30, no. 6, pp. 2574–2587, Dec. 2022.
- [11] X. Cai, "Website fingerprinting attacks and defenses on anonymity networks," Ph.D. dissertation, Dept. Comput. Sci., State Univ. New York Strong Brook, Strong Brook, NY, USA, 2014.

- [12] T. Wang, "Website fingerprinting: Attacks and defenses," Ph.D. dissertation, Dept. Comput. Sci., Univ. Waterloo, Waterloo, ON, Canada, 2016.
- [13] P. Liu, L. He, and Z. Li, "A survey on deep learning for website fingerprinting attacks and defenses," *IEEE Access*, vol. 11, pp. 26033–26047, 2023.
- [14] F. Zaki, F. Afifi, S. Abd Razak, A. Gani, and N. B. Anuar, "GRAIN: Granular multi-label encrypted traffic classification using classifier chain," *Comput. Netw.*, vol. 213, Aug. 2022, Art. no. 109084.
- [15] H. Zhang, X. Xiao, L. Yu, Q. Li, Z. Ling, and Y. Zhang, "One train for two tasks: An encrypted traffic classification framework using supervised contrastive learning," 2024, *arXiv:2402.07501*.
- [16] D. Herrmann, R. Wendolsky, and H. Federrath, "Website fingerprinting: Attacking popular privacy enhancing technologies with the multinomial Naïve-bayes classifier," in *Proc. ACM Workshop Cloud Comput. Secur.*, Nov. 2009, pp. 31–42.
- [17] X. Cai, X. C. Zhang, B. Joshi, and R. Johnson, "Touching from a distance: Website fingerprinting attacks and defenses," in *Proc. ACM Conf. Comput. Commun. Secur.*, Oct. 2012, pp. 103–114.
- [18] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *Proc. USENIX Secur. Symp.*, Aug. 2014, pp. 143–157.
- [19] A. Panchenko et al., "Website fingerprinting at internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 263–274.
- [20] J. Hayes and G. Danezis, "K-fingerprinting: A robust scalable website fingerprinting technique," in *Proc. USENIX Secur. Symp.*, Jan. 2015, pp. 1187–1203.
- [21] K. Abe and S. Goto, "Fingerprinting attack on tor anonymity using deep learning," in *Proc. Asia-Pacific Adv. Netw. Res. Workshop*, vol. 42, Jan. 2016, pp. 15–20.
- [22] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, "WF-transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2024.
- [23] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
- [24] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM Comput. Surveys*, vol. 53, no. 3, pp. 1–34, Jun. 2020.
- [25] C. Liu et al., "Overcoming data limitations: A few-shot specific emitter identification method using self-supervised learning and adversarial augmentation," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 500–513, 2023.
- [26] M. Juárez, M. Imani, M. Perry, C. Díaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. Eur. Symp. Res. Comput. Secur.*, Jan. 2016, pp. 27–46.
- [27] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *Proc. 29th USENIX Conf. Secur. Symp.*, Jan. 2020, pp. 717–734.
- [28] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 227–238.
- [29] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.
- [30] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, vol. 119, Jul. 2020, pp. 1597–1607.
- [31] A. Papandreou-Suppappola, Ed., *Applications in Time-Frequency Signal Processing*. Boca Raton, FL, USA: CRC Press, 2003.
- [32] M. Dakovic, L. Stankovic, and T. Thayaparan, *Time-Frequency Signal Analysis With Applications*. Boston, MA, USA: Artech House, 2013.
- [33] E. O. Brigham and R. E. Morrow, "The fast Fourier transform," *IEEE Spectr.*, vol. S-4, no. 12, pp. 63–70, Dec. 1967, doi: [10.1109/MSPEC.1967.5217220](https://doi.org/10.1109/MSPEC.1967.5217220).
- [34] M. Juárez, S. Afroz, G. Acar, C. Diaz, and R. Greenstadt, "A critical evaluation of website fingerprinting attacks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 263–274.
- [35] P. Sirinam, N. Mathews, M. S. Rahman, and M. K. Wright, "Triplet fingerprinting: More practical and portable website fingerprinting with N-shot learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1131–1148.
- [36] E. Hoffer and N. Ailon, "Deep metric learning using triplet network," in *Proc. Int. Workshop Similarity-Based Pattern Recognit.*, Dec. 2014, pp. 84–92.
- [37] M. Chen, Y. Wang, Z. Qin, and X. Zhu, "Few-shot website fingerprinting attack with data augmentation," *Secur. Commun. Netw.*, vol. 2021, pp. 1–13, Sep. 2021.
- [38] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient website fingerprinting attack based on deep learning," *Proc. Privacy Enhancing Technol.*, vol. 2019, no. 4, pp. 292–310, Oct. 2019.
- [39] A. van den Oord et al., "WaveNet: A generative model for raw audio," in *Proc. Speech Synth. Workshop*, Jan. 2016, pp. 1–5.
- [40] M. Chen, Y. Wang, H. Xu, and X. Zhu, "Few-shot website fingerprinting attack," *Comput. Netw.*, vol. 198, no. 4, Oct. 2021, Art. no. 108298.
- [41] Z. Lu, L. Song, Z. Han, Z. Ma, and X. Wang, "Multi-temporal domain feature adaptation representation framework for website fingerprinting," in *Proc. 3rd Int. Conf. Mach. Learn.*, Springer, F. Sun, H. Wang, H. Long, Y. Wei, and H. Yu, Eds., Singapore: Springer, Jan. 2025, pp. 127–137.
- [42] J. Tan, H. Wang, S. Han, D. Man, and W. Yang, "An adaptability-enhanced few-shot website fingerprinting attack based on collusion," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8220–8235, 2024.
- [43] G. Cherubin, R. Jansen, and C. Troncoso, "Online website fingerprinting: Evaluating website fingerprinting attacks on tor in the real world," in *Proc. 31st USENIX Secur. Symp.*, Aug. 2022, pp. 753–770.
- [44] A. Bahramali, A. Bozorgi, and A. Houmansadr, "Realistic website fingerprinting by augmenting network traces," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 1035–1049.
- [45] Y. Ding and B. Hu, "A multi-granularity features representation and dimensionality reduction network for website fingerprinting," *IEEE Access*, vol. 13, pp. 574–587, 2025, doi: [10.1109/ACCESS.2024.3522892](https://doi.org/10.1109/ACCESS.2024.3522892).
- [46] M. Caron, P. Bojanowski, J. Mairal, and A. Joulin, "Unsupervised pre-training of image features on non-curved data," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 2959–2968.
- [47] Z. Dai, B. Cai, Y. Lin, and J. Chen, "UP-DETR: Unsupervised pre-training for object detection with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 1601–1610.
- [48] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, vol. 1, Jan. 2018, pp. 4171–4186.
- [49] R. Christensen, *Frequency Analysis of Time Series*. New York, NY, USA: Springer, 2001, pp. 152–199.
- [50] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.
- [51] W. S. Gan, *Discrete Fourier Transform*. Singapore: Springer, 2020, pp. 13–16.
- [52] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, vol. 1. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>
- [53] L. V. D. Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, Nov. 2008.